Let the Solver Choose Its Data:

Likelihood-Band Selection of Synthetic ARC-AGI-2 Tasks, and Why Held-Out Loss Is the Wrong Signal

Harthik Mallichetty
Independent researcher, Dallas, TX
harthikusa@gmail.com Kaggle: harthikrm

ARC Prize 2026 Paper Track · November 2026

Abstract

Large synthetic task pools now exist for the Abstraction and Reasoning Corpus, but most of their contents are either already solved by a strong solver or unlearnable by it. We propose *solver-in-the-loop* curation: score every candidate task with the target solver's own zero-shot teacher-forced likelihood of the answer (one forward pass, no decoding, no labels beyond the task itself), and keep only the tasks whose per-token negative log-likelihood (NLL) falls inside the band spanned by the target benchmark's 25th–95th percentiles and that the solver does not already solve greedily. Applied to ARC-Mega [6], the method scores 114,067 tasks in ≈8.4 hours on Kaggle's 4×L4 machine and retains 42%. Continue-training the 2025-winning NVARC model [19] on only 6,000 selected tasks (0.008% of the pool's ARC tasks, 2 GPU-hours on one H200) raised its hidden-test score on the ARC Prize 2026 leaderboard from a mean of 28.5 (three draws of the stock checkpoint) to 31.2 (two draws), with the unchanged test-time-training pipeline. Scaling the same recipe to 47,435 tasks lowered the held-out loss on selected tasks by 39% but *did not* improve the hidden score (29.6, one draw), while a forgetting monitor on the public evaluation set turned upward. We argue that the band works because it targets the solver's learning frontier, that in-distribution held-out loss is the wrong model-selection signal once the solver adapts per task at test time, and that a rise in the forgetting monitor is the earlier and cheaper warning. All code is released; the method needs only a solver that assigns likelihoods and is agnostic to how the pool was generated.

1 Introduction

ARC-AGI-2 [5] tests whether a system can infer a novel grid transformation from a few demonstrations. The 2025 Kaggle winner, NVARC [19], is a transductive pipeline: a Qwen3-4B model [17] fine-tuned on ∼100k LLM-generated puzzles, adapted to each test task with LoRA test-time training (TTT) [1], decoded with depth-first search, and re-scored under augmentations [7, 8]. Its public code and checkpoint became the starting point for a large fraction of the 2026 field.

In September 2026, MindsAI/Tufa Labs released ARC-Mega, a 134 GB pool of roughly 97 million training rows, about 74 million of them ARC tasks [6]. The pool is far larger than any single team can train on under a competition budget, and it is heterogeneous: our probe (§4) finds that 14.5% of its profile-matched tasks are already solved greedily by NVARC without any adaptation and that a further 10% carry answer likelihoods below the benchmark's own 95th-percentile tail. The practical question for a resource-limited competitor is therefore not *how to generate more data* but *which of the abundant data to spend compute on*.

This paper answers that question with a method that needs nothing beyond the solver itself (Figure 1). We compute, for each candidate task, the per-token NLL that the frozen solver assigns to the correct test output given the demonstrations, and we keep the task if (i) the solver does not already reproduce the output

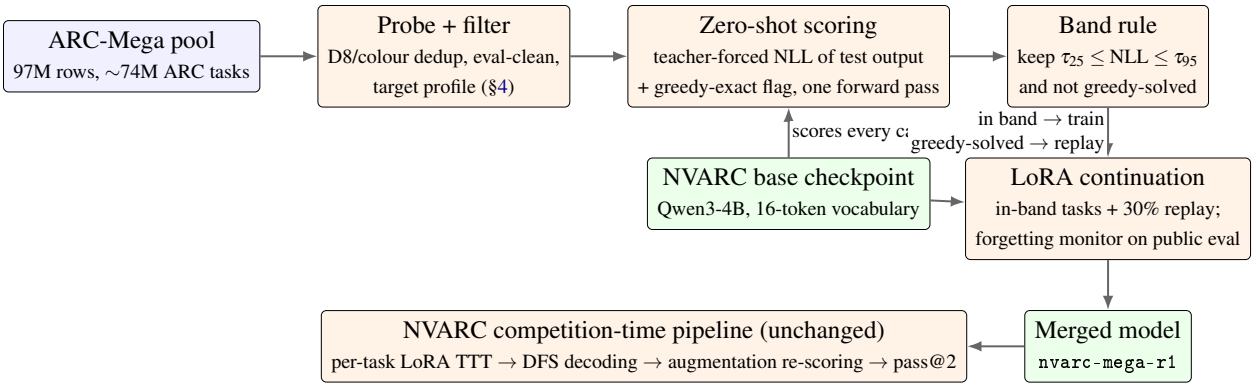


Figure 1: Solver-in-the-loop curation. The frozen solver scores every candidate once; the band rule keeps tasks at the solver's learning frontier; greedy-solved tasks become replay. Only the base checkpoint changes; the competition-time pipeline is the public NVARC code.

under greedy decoding and (ii) its NLL lies within the 25th–95th percentile band of the solver's NLL on the public ARC-AGI-2 evaluation set. Everything the solver already solves is kept as replay [18]. The selected tasks are used for a short LoRA continuation of the solver's base checkpoint; the rest of the NVARC pipeline is untouched.

Contributions.

1. A label-free, one-forward-pass data-curation rule that matches a synthetic pool to a target benchmark *through the solver's eyes*, together with a characterisation of ARC-Mega under it (§3–§4).
2. Hidden-leaderboard evidence that 6k selected tasks improve NVARC by +2.7 points on ARC Prize 2026 (§5); the linked Kaggle submission is this model.
3. A negative scaling result: 8× more selected data lowers the in-distribution held-out loss by 39% yet does not improve the hidden score. We explain why held-out loss on the selected pool is the wrong signal once the solver adapts per task, and show that a forgetting monitor on the target benchmark flags the problem earlier (§6).
4. A record of three further levers we measured and rejected under the same budget—answer re-ranking, CPU program search, and test-time program induction with 30B-class open models—so that others do not repeat them (§7).

2 Background and related work

ARC and transductive solvers. ARC [4] tasks present 2–7 demonstration pairs of coloured grids and one or two test inputs; a prediction is correct only if every cell matches, with two attempts per test input (pass@2). The dominant Kaggle approach since 2024 fine-tunes a small language model on grid-to-grid examples and adapts it per task at test time [1, 7, 8]. NVARC [19] added a four-stage synthetic data pipeline (puzzle descriptions from H-ARC and BARC, LLM-mixed descriptions, generated input programs, consistency-filtered output programs) and full fine-tuning on 3.2M augmented samples. Its authors report that validation loss on augmented public-evaluation puzzles correlated with the public leaderboard; our results qualify that observation (§6). Inductive (program-synthesis) solvers [9, 13, 22] and recursive tiny models [11] are complementary; we measured both under our budget (§7).

Synthetic ARC data. Re-ARC [9] provides generators for the 400 ARC-AGI-1 training tasks; BARC [13] and NVARC generate new tasks with LLM-written programs; ARC-Mega [6] aggregates MindsAI training

data at a scale (97M training rows, ∼74M ARC tasks) two orders of magnitude beyond prior releases. Prior work asks how to *generate* such pools; we ask how to *select* from one when training compute is the binding constraint.

Loss-based data selection. Selecting examples by a model's own loss has a long history: curriculum learning [3], RHO-LOSS, which prioritises points that are "learnable, worth learning, and not yet learnt" [15], dataset cartography [21], and perplexity-based pruning for LLM pre-training [2, 14]. Sorscher et al. [20] show that in the data-abundant regime one should keep *hard* examples but that the hardest fraction is harmful. Gradient-based selection [23] targets a downstream distribution at a higher cost. Our rule is a cheap instance of this family specialised to the ARC setting: the *target solver* is the reference model, the difficulty band is anchored to the *target benchmark's* percentiles rather than to fixed thresholds, and "already learnt" is operationalised by greedy exact-match, which is the actual success criterion for ARC.

3 Method

3.1 Zero-shot likelihood scoring

Let S_θ be the frozen solver and let a task t consist of demonstrations $D = \{(x_i, y_i)\}$ and a test pair (x^*, y^*). We serialise (D, x^*) in NVARC's dialogue format (each grid is rows of digits; user turns carry inputs, assistant turns carry outputs; the tokenizer has 16 tokens) and run a single teacher-forced forward pass to obtain the answer's token log-probabilities. We record

$$\mathrm{NLL}(t) = -\frac{1}{|y^*|}\sum_{j=1}^{|y^*|} \log p_\theta\left(y^*_j \mid D, x^*, y^*_{<j}\right), \qquad \mathrm{greedy}(t) = \mathbb{1}\left[\forall j:\ y^*_j = \arg\max_v p_\theta(v \mid D, x^*, y^*_{<j})\right]. \quad (1)$$

$\mathrm{greedy}(t) = 1$ means greedy decoding would reproduce the answer exactly, i.e. the task is already solved without adaptation. If the serialised task exceeds 8,192 tokens we drop leading demonstrations until it fits (down to one); tasks that still do not fit are skipped. No generation, search or test-time training is involved, so scoring costs one forward pass per task: ≈13,600 tasks per hour on Kaggle's 4×L4 (92,388 tasks in 6.8 h).

3.2 The band rule

We score the public ARC-AGI-2 evaluation set (120 tasks, 172 test outputs) with the same procedure and take its 25th and 95th per-token-NLL percentiles, $\tau_{25} = 0.0324$ and $\tau_{95} = 0.3265$ nats/token. A candidate task is *selected* if

$$\mathrm{greedy}(t) = 0 \quad \text{and} \quad \tau_{25} \le \mathrm{NLL}(t) \le \tau_{95}. \quad (2)$$

Candidates with $\mathrm{greedy}(t) = 1$ form the *replay* set, together with the 1,000 public training tasks. Candidates below τ_{25} are considered too easy to teach anything the solver's TTT cannot already recover; candidates above τ_{95} are considered unlearnable or malformed for this solver. The lower cut is the one that matters for efficiency (33% of the pool, Figure 3a); the upper cut mainly removes a distinct population of tasks with very short outputs (§6). A fixed, disjoint set of 300 selected tasks is held out as the in-distribution validation set for every run.

Why anchor to the benchmark's percentiles rather than fixed thresholds. The same per-token NLL means different things for different solvers and formats. Anchoring the band to the target benchmark makes the rule portable: for a new solver, score the benchmark once and the thresholds follow. It also makes the rule *conservative by construction*: it cannot select tasks the solver finds harder than the benchmark's own hard tail.

Algorithm 1 Solver-in-the-loop band selection

Require: frozen solver S_θ; target benchmark B; candidate pool P; quantiles $(q_{lo}, q_{hi}) = (0.25, 0.95)$

$\{\mathrm{NLL}(b)\}_{b\in B} \leftarrow$ teacher-forced scoring of B with S_θ; $\tau_{lo}, \tau_{hi} \leftarrow$ quantiles q_{lo}, q_{hi}
$T \leftarrow \emptyset$, $R \leftarrow$ public training tasks
for all $t \in P$ (first demonstration input deduplicated under D8+colour; no grid shared with B) **do**
    compute $\mathrm{NLL}(t)$ and $\mathrm{greedy}(t)$ in one forward pass
    if $\mathrm{greedy}(t)$ **then** $R \leftarrow R \cup \{t\}$ ▷ already solved: replay
    else if $\tau_{lo} \le \mathrm{NLL}(t) \le \tau_{hi}$ **then** $T \leftarrow T \cup \{t\}$ ▷ learning frontier: train
    end if
end for
continue-train S_θ on T with 30% replay from R; log held-out loss on T_{val} and forgetting monitor on B

3.3 Continued training with a forgetting monitor

We continue-train the NVARC base checkpoint (`sorokin/qwen3_4b_grids15_sft139`) with rank-stabilised LoRA [10, 12] (r=128, α=32) on all attention and MLP projections, learning rate 5×10^{-5} with cosine decay and 3% warm-up, global batch 32, bf16, maximum length 8,192, loss on output grids only, and NVARC-style augmentation (random D8 transform, full colour permutation, demonstration shuffle; one augmentation per task per epoch). Thirty percent of training samples are drawn from the replay set. Training data order is a deterministic function of the sample index, so runs resume exactly across providers.

Two losses are logged at step 0 and every 200 steps: (a) the *held-out Mega loss* on the 300 held-out selected tasks, the in-distribution signal one would normally use for model selection; and (b) the *forgetting monitor*, the loss on the public ARC-AGI-2 evaluation set. The base checkpoint was trained on the public evaluation puzzles (NVARC's "full" subset includes them [19]), so the *level* of (b) carries little information, but a *rise* indicates that the continuation is moving the model away from what it already knew. After training the adapter is merged into the base weights and uploaded as a Kaggle model; the competition notebook is the public NVARC notebook with the model path changed and a deadline-safe scaffold (Appendix A).

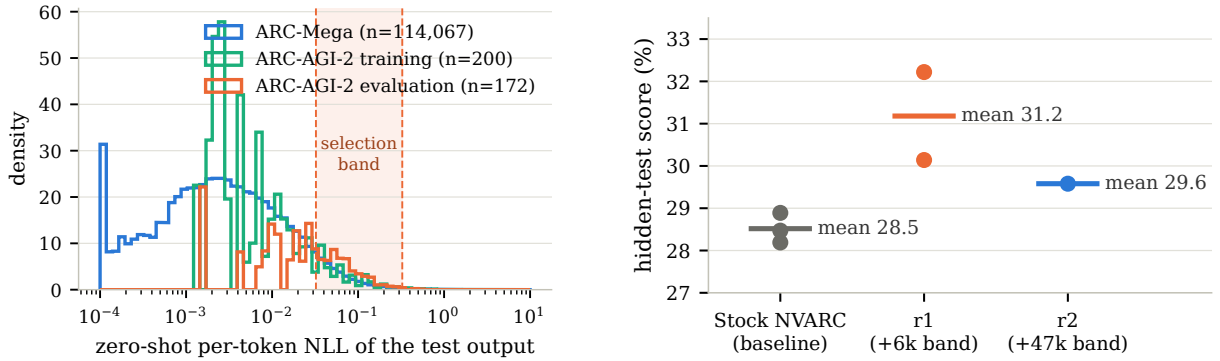
4 Characterising ARC-Mega through the solver

Probe. We did not download the full 134 GB. A probe script samples 39 of the 190 training shards at even spacing plus the validation and test splits (capped at 20,000 tasks per shard), recognises ARC-format records, and applies three filters: (i) *deduplication* by a hash invariant to the eight dihedral symmetries and to colour relabelling, (ii) an *evaluation firewall* that rejects any task sharing a grid (under D8) with the 120 public evaluation tasks, and (iii) a *target profile*—first demonstration input with $\ge$5 colours, $\ge$300 cells and $\ge$3 connected objects—chosen because NVARC misses are concentrated on larger, more colourful grids (public-evaluation inputs average 5.4 colours and 351 cells versus 3.8 colours and 161 cells for public-training tasks). Table 1 summarises what the probe found. Two facts matter for anyone using the pool: the 166-task "test" split *is* the public ARC-AGI-2 evaluation set (100% grid overlap), so it is not a clean held-out set; and only 77% of training rows are distinct under augmentation.

Scoring. Two probe batches kept 21,753 and 92,681 distinct, eval-clean, profile-matched tasks, of which 21,679 and 92,388 (114,067 in total) were scored zero-shot; the remainder exceeded the 8,192-token limit or the scoring deadline. Figure 2a compares the per-token NLL distributions. NVARC solves 14.5% of these ARC-Mega tasks greedily versus 18.0% of a 200-task public-training sample and 1.2% (2 of 172 outputs) of the public evaluation set; the ARC-Mega median NLL (0.036) is roughly half the evaluation median (0.067),

Table 1: ARC-Mega under the probe (sampled shards). "Distinct" is modulo D8 and colour permutation. The 166-task test split coincides with the public evaluation set.

Split	Tasks sampled	Distinct	Overlap w/ public eval	Overlap w/ public train	Target profile
test	166	114	166 (100%)	0	63 (38%)
validation	9,748	9,457	20 (0.2%)	1,618	1,580 (16%)
train	780,000	599,026	1,726 (0.2%)	138,485	126,368 (16%)



(a) Per-token NLL under the frozen NVARC model (log-x). Shaded: the selection band, the evaluation set's P25–P95.

(b) Hidden-test scores on the ARC Prize 2026 public leaderboard (≈120 hidden tasks). Each dot is one full 12-hour scoring run.

Figure 2: Where the solver finds ARC-Mega hard (left) and what training on the hard band did (right).

and its distribution is much wider (5th–95th percentiles 0.0025–0.61 versus 0.011–0.33). Figure 3a shows the resulting partition: 16,505 tasks (14%) become replay, 38,031 (33%) are below the band, 47,735 (42%) are selected, and 11,796 (10%) exceed the upper cut. Figure 3b shows that the greedy-exact rate collapses from 80% in the lowest NLL bin to ≈1–2% throughout the band, so the lower cut and the greedy flag are largely redundant above τ_{25} and the flag is what rescues the few in-band tasks that happen to be solved.

5 Experiments on the hidden test set

Setup. All scores are ARC Prize 2026 public-leaderboard scores: pass@2 exact match on ≈120 hidden tasks, computed by a 12-hour re-run of the submitted notebook on 4×L4 with no internet. Because Kaggle allows one submission per day and the pipeline is stochastic (batched DFS decoding is not batch-invariant [19]), each configuration was scored as many times as the budget allowed. The three stock-checkpoint draws, made with successive versions of the same deadline scaffold (v6–v8), spread by 0.7 points; we treat ±1.5 as the working noise level. Local evaluation accuracy is not reported because the base checkpoint was trained on the public evaluation set (§3); the hidden draws are the only clean signal we have.

Runs. Table 2 lists the two continuation runs. **r1** trains on 6,000 selected tasks from the first probe batch (plus replay) for one epoch, ≈270 optimizer steps, 2.05 h on one H200. **r2** trains on all 47,435 selected tasks outside the validation split, 2,118 steps, 11.7 h on one RTX PRO 6000 (96 GB), same hyper-parameters. Both are merged and dropped into the unchanged NVARC notebook.

Result 1: a small selected set helps. r1 scored 32.22 and 30.14 on two independent scoring runs against a baseline mean of 28.5 (Figure 2b): +2.7 points from 6,000 tasks, 0.008% of the pool's ARC tasks and 0.2% of the samples NVARC was originally trained on, at a cost of two GPU-hours. The lowest r1 draw exceeds

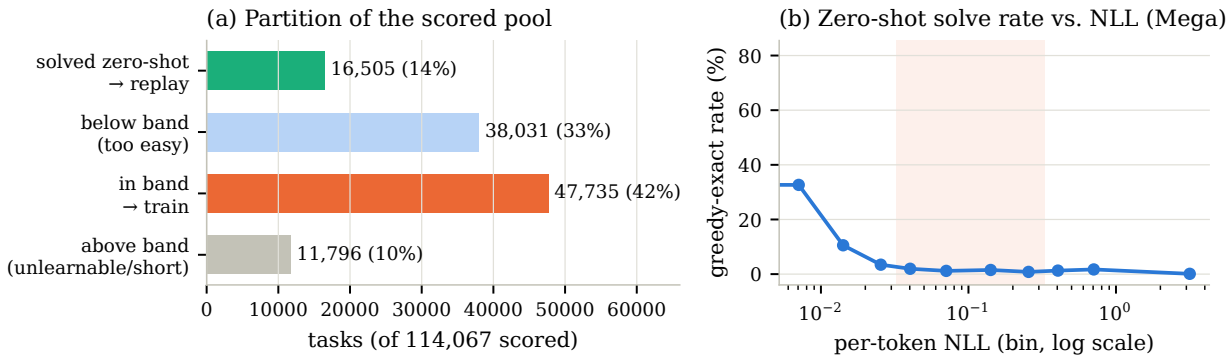


Figure 3: (a) The band rule partitions the 114,067 scored tasks. (b) Greedy-exact rate by NLL bin on ARC-Mega: the band (shaded) sits where almost nothing is solved zero-shot yet token accuracy is still high (92–99%).

Table 2: Continuation runs and outcomes. Losses are nats per output token, step 0 → end (r2 also at step 800). Hidden-test scores are public-leaderboard draws of the same notebook; only the checkpoint differs.

Run	Selected	Steps	GPU-h	Held-out loss (Mega val.)	Forgetting monitor (eval)	Hidden-test draws (mean)
baseline	—	—	—	0.0734	0.0724	28.89, 28.47, 28.19 (28.5)
r1	6,000	≈270	2.05	0.0734 → 0.0603 (−18%)	0.0724 → 0.0692	**32.22, 30.14 (31.2)**
r2	47,435	2,118	11.7	0.0734 → 0.0450 (−39%)	0.0724 → 0.0707 → 0.0716	29.58 (29.6)

the highest baseline draw by 1.25 points. With two and three draws the difference is suggestive rather than statistically settled (Welch $t \approx 2.5$ on one effective degree of freedom); we report it as such. The r1 model is the linked submission.

Result 2: scaling the selected set did not help. r2 reached a held-out Mega loss 39% below the start (Figure 4)—more than twice r1's improvement on exactly the same 300 validation tasks—yet scored 29.58, below both r1 draws and within noise of the baseline. Had we selected the model by the in-distribution validation loss, as is standard and as NVARC's authors did with public-evaluation loss, we would have shipped the worse model. The forgetting monitor told a different story: it stood at 0.0707 at step 800 and ended at 0.0716, i.e. it rose over the last 1,300 steps of r2, while r1 ended at 0.0692.

6 Analysis: why the band works, and why its own loss misleads

What the band selects. Figure 5 profiles the three NLL regions of the first probe batch against the evaluation set. Tasks below the band are dominated by large outputs that copy most of the input (83% same-shape, median 504 answer tokens, 99% token accuracy): the per-token NLL is low because most tokens are trivially predictable, not because the rule is known. Tasks above the band are a distinct population with very short outputs (median 30 tokens, 10% same-shape): with few copyable tokens every token is a decision and the per-token NLL is high, so the upper cut is largely a *length* filter rather than a difficulty filter. The band itself matches the evaluation set on token accuracy (97% vs. 97%) and answer length (270 vs. 389 tokens) more closely than either tail, which is what anchoring to the benchmark's percentiles is meant to achieve. A length-aware score (e.g. NLL restricted to tokens that differ from the input, or a per-task z-score) would make the upper cut a genuine difficulty cut; we leave this to future work.

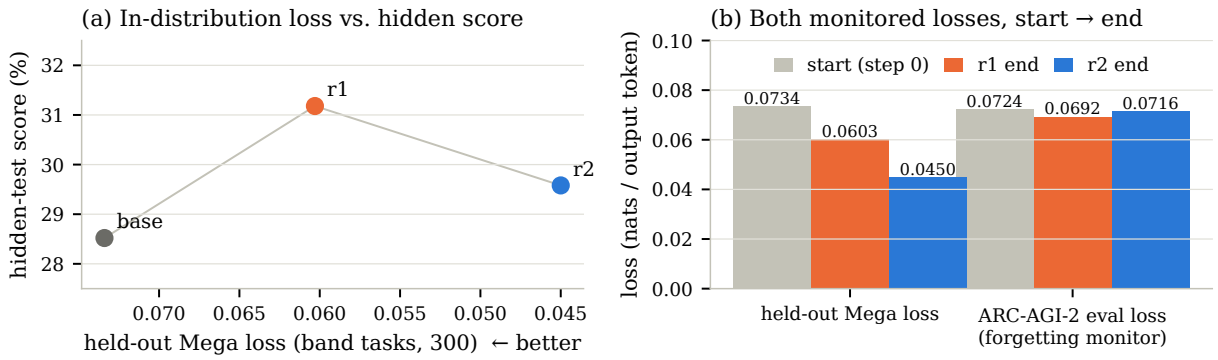


Figure 4: (a) Held-out loss on the selected pool versus hidden-test score: the in-distribution loss keeps improving from r1 to r2 while the hidden score falls back. (b) Both monitored losses at the start and end of each run; only the forgetting monitor orders the runs the way the hidden test does.

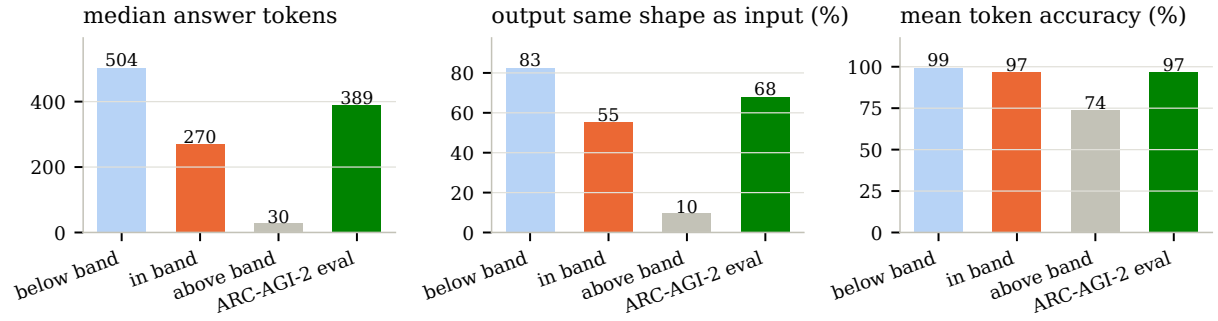


Figure 5: What the band rule selects (first probe batch, 21,679 tasks) compared with the public evaluation set. Below-band tasks are long, copy-heavy outputs; above-band tasks are very short outputs; the band resembles the evaluation set.

Why a small selected set helps. NVARC adapts to every test task with LoRA TTT, so the base checkpoint's job is not to *know* any particular task but to supply a good prior over rule families and grid regularities from which a few hundred TTT steps can recover the specific rule. The band contains tasks the solver nearly handles (token accuracy 92–99%) but does not quite (greedy-exact 1–2%): in the vocabulary of Mindermann et al. [15], learnable and not yet learnt. A short continuation on such tasks adjusts the prior where it is weakest, and the forgetting monitor confirms that the base knowledge is not disturbed (0.0724 → 0.0692).

Why in-distribution held-out loss is the wrong signal. The held-out Mega loss measures fit to the *generator distribution* of the selected pool. That distribution differs from ARC-AGI-2 in ways that the band cannot remove: it is produced by a finite family of program generators, 23% of its rows are augmented copies, and its NLL distribution is wider than the benchmark's. With 8× more steps at the same learning rate, r2 travels much further from the NVARC optimum and learns Mega-specific regularities that lower Mega loss without improving the prior that TTT needs; the hidden score is flat and the forgetting monitor rises over the last 1,300 steps of the run. This is a Goodhart effect specific to solvers with test-time adaptation: the quantity that TTT will fix anyway (task-specific likelihood) is exactly the quantity the validation loss rewards. NVARC's observation that evaluation-set loss tracked the leaderboard was made during *pre*-training on a broad, benchmark-matched synthetic distribution; our result is that the same proxy fails for *continuation* on a narrower external pool.

The forgetting monitor as an early-stopping signal. Across our runs the ordering of final hidden scores (r1 $>$ r2 $\approx$ baseline) is the ordering of final forgetting-monitor values ($0.0692 < 0.0716 < 0.0724$), and is the *reverse* of the ordering by held-out Mega loss. The monitor costs one evaluation pass over 120 public tasks every 200 steps. We therefore recommend, for any continuation of a TTT-based solver: stop at the first upward turn of the target-benchmark loss, and ignore the in-distribution validation loss for model selection. A noisy-but-clean hidden draw remains the final arbiter.

What a practitioner should take from this. (i) Score the pool with the solver before training on it; one forward pass per task is cheap enough to do for 10^5–10^6 tasks on competition hardware. (ii) Train briefly on the band, replay what is already solved. (iii) Watch the target benchmark's loss for a rise, not the pool's loss for a fall. (iv) Expect single-digit-thousand selected tasks to be enough; more is not better without a second lever (lower learning rate, mixed sources, or a length-aware score).

7 Levers we measured and rejected

Under the same solo budget we evaluated three other routes to a higher score. Each was measured on the public evaluation set with paired per-task comparisons against the stock NVARC run; we report them so that the completeness criterion is met and so that others can skip them.

- **Answer re-ranking.** NVARC generates candidate grids by DFS and ranks them by an augmentation-consistency score. On the 120 public evaluation tasks the stock rule scores 33.3 task-points; a perfect oracle over the same candidates would score 38.5 (54 of 172 outputs have a correct candidate). The best of 14 alternative rules (`probmul_3`, which sums candidate and augmentation log-probabilities) recovered only $+0.5$ task-points: in 8 of 9 recoverable misses the wrong answer out-scored the right one on *every* signal. The headroom is real but the model is confidently wrong, so re-ranking cannot reach it.
- **CPU program search.** The 2020 winning DSL search [22] at depth 3 solved 2.5 task-points on the evaluation set in 55 CPU-minutes, only 0.5 beyond NVARC; worth filling empty attempt slots, not worth displacing a transductive attempt.
- **Test-time program induction with open 30B-class models.** We built a multi-round propose–execute–verify loop in the exact tool format of NVIDIA's ARC agentic traces [16], with a verifier that checks demonstration pass, leave-one-out generalisation, literal-grid hygiene and cross-program consensus. On 57 development tasks the best zero-shot model (Qwen3.6-35B-A3B) produced final programs that passed all demonstrations in 2 of 122 cases, truncated 37% of its reasoning turns, and solved nothing NVARC missed; in a preceding 5-task smoke test two other models (Nemotron-3-Nano-30B, Qwen3-30B-Thinking) truncated 98–100% of their turns and produced no programs. Without fine-tuning on such traces, induction at this model size is not competitive inside 12 hours on L4s. A reusable finding from this branch: verifiers also have *false negatives*—correct programs were rejected because the model's own test code embedded demonstration grids—fixed by verifying only the code reachable from the solution function.

8 Limitations

Our evidence is six hidden-test draws across three configurations with a noise floor of about ± 1.5 points; the $+2.7$ effect is consistent across both r1 draws but is not established beyond doubt. We ran one base model, one seed per configuration and one learning rate, so we cannot separate "too much data" from "too many steps at this learning rate"; a 6k run with random rather than band selection is the control we most wanted and could not afford before the deadline. The upper cut is partly a length filter (§6). Public-leaderboard scores use about half of the hidden tasks; private scores may differ. Finally, the target profile used in the probe was

chosen by inspection of NVARC misses and is a confound: the band was tested only on profile-matched tasks.

9 Conclusion

A strong ARC solver can tell you which synthetic tasks are worth its training time: score the pool once with the solver's own likelihood, keep what lies in the benchmark's difficulty band and is not yet solved, replay what is. Six thousand such tasks and two GPU-hours moved the 2025 winner by $+2.7$ points on the 2026 hidden test set; forty-seven thousand did not move it further, even as the in-distribution loss fell by 39%. The method requires nothing but a likelihood-assigning solver and a target benchmark, so it transfers directly to any few-shot induction setting with a synthetic pool; the lesson about validation signals transfers to any solver that adapts at test time.

Reproducibility. Code (probe, scoring notebook, data builder, training kit, selection lab) and the linked Kaggle notebook and model are listed in Appendix A.

Acknowledgements. We thank the NVARC authors for releasing their code and checkpoint, and MindsAI/Tufa Labs for releasing ARC-Mega.

References

[1] Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for abstract reasoning. *arXiv preprint arXiv:2411.07279*, 2024.

[2] Zachary Ankner, Cody Blakeney, Kartik Sreenivasan, Max Marion, Matthew L. Leavitt, and Mansheej Paul. Perplexed by perplexity: Perplexity-based data pruning with small reference models. *arXiv preprint arXiv:2405.20541*, 2024.

[3] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning (ICML)*, 2009.

[4] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[5] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.

[6] Jack Cole. ARC Mega dataset. Hugging Face dataset, https://huggingface.co/datasets/mindware/arc-agi-mega, 2025.

[7] Daniel Franzen, Jan Disselhoff, and David Hartmann. The LLM ARChitect: Solving ARC-AGI is a matter of perspective. ARC Prize 2024 1st-place technical report, https://github.com/da-fr/arc-prize-2024/blob/main/the_architects.pdf, 2024.

[8] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. *arXiv preprint arXiv:2505.07859*, 2025.

[9] Michael Hodel. Addressing the abstraction and reasoning corpus via procedural example generation. *arXiv preprint arXiv:2404.07353*, 2024.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[11] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.

[12] Damjan Kalajdzievski. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*, 2023.

[13] Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, Wei-Long Zheng, Zenna Tavares, Yewen Pu, and Kevin Ellis. Combining induction and transduction for abstract reasoning. *arXiv preprint arXiv:2411.02272*, 2024.

[14] Max Marion, Ahmet Üstün, Luiza Pozzobon, Alex Wang, Marzieh Fadaee, and Sara Hooker. When less is more: Investigating data pruning for pretraining LLMs at scale. *arXiv preprint arXiv:2309.04564*, 2023.

[15] Sören Mindermann, Jan M. Brauner, Muhammed T. Razzak, Mrinank Sharma, Andreas Kirsch, Winnie Xu, Benedikt Höltgen, Aidan N. Gomez, Adrien Morisot, Sebastian Farquhar, and Yarin Gal. Prioritized training on points that are learnable, worth learning, and not yet learnt. In *International Conference on Machine Learning (ICML)*, 2022.

[16] NVIDIA. Nemotron-SFT-ARC-AGI-v1. Hugging Face dataset, https://huggingface.co/datasets/nvidia/Nemotron-SFT-ARC-AGI-v1, 2026.

[17] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[18] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[19] Ivan Sorokin and Jean-François (CPMP) Puget. NVARC solution: Synthetic data generation with LLMs and code generation + The ARChitects + TRM. ARC Prize 2025 1st-place solution write-up, https://www.kaggle.com/competitions/arc-prize-2025/writeups/nvarc; code at https://github.com/1ytic/NVARC, 2025.

[20] Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari S. Morcos. Beyond neural scaling laws: Beating power law scaling via data pruning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[21] Swabha Swayamdipta, Roy Schwartz, Nicholas Lourie, Yizhong Wang, Hannaneh Hajishirzi, Noah A. Smith, and Yejin Choi. Dataset cartography: Mapping and diagnosing datasets with training dynamics. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[22] Johan Sokrates Wind. ARC 2020 1st place solution. Kaggle Abstraction and Reasoning Challenge, https://github.com/top-quarks/ARC-solution, 2020.

[23] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. LESS: Selecting influential data for targeted instruction tuning. In *International Conference on Machine Learning (ICML)*, 2024.

A Reproducibility details

Linked submission. Kaggle notebook: [notebook URL].
Submission ID: [ID] (model `harthikrm/nvarc-mega-r1`; public-leaderboard score 32.22, second draw 30.14). Code: [GitHub URL].

Competition-time pipeline. The public NVARC notebook (pinned Kaggle environment of 2025-07-07, Unsloth with the flash-attention patch) with three changes: a glob that resolves the model directory (`NVARC_MODEL_DIR`); a placeholder `submission.json` written in the first minute covering all hidden task IDs; and a `StopAtDeadline` callback that halts test-time training at 11.75 h so the 12-hour limit cannot zero the run. Per-task TTT (LoRA r=256, α=32, bf16), batched DFS decoding with a per-task cap, and the stock augmentation re-scoring rule are unchanged.

Scoring notebook. Four worker processes (one per L4), teacher-forced forward passes over length-sorted batches with a 24,576-token budget, right padding, bf16, SDPA attention; outputs `mega_scores.csv` with per-output `nll_mean`, `nll_sum`, `greedy_exact`, `frac_tokens_correct` and `n_train_used`. Throughput 13.6k tasks/h.

Probe.

```
mega_probe.py -max-gb 15 -max-tasks-per-file 20000 -exclude <earlier batch>
```

Dedup hash: minimum over D8 of a blake2b digest of the colour-rank-relabelled grid; evaluation firewall: any demonstration grid or test input with more than nine cells that matches a public-evaluation grid under D8 rejects the task; dedup uses the first demonstration input; target profile: $\geq$5 colours, $\geq$300 cells, $\geq$3 four-connected non-background components in the first demonstration input.

Data builder.

```
build_train_data.py -mega mega_sample [-mega mega_big]
  -eval-quantile 0.25 -eval-upper-quantile 0.95 -max-train 6000|60000
  -n-val 300 -reuse-val train_data/val.json
```

Thresholds computed from the 172 evaluation outputs: $\tau_{25} = 0.03236$, $\tau_{95} = 0.32646$. Replay = greedy-solved Mega tasks + 1,000 public training tasks.

Training.

```
train_lora.py -rank 128 -alpha 32 -lr 5e-5 -warmup 0.03 -global-batch 32
  -max-len 8192 -n-aug 1 -replay-frac 0.3 -eval-steps 200
  -arc-eval <public eval>
```

rsLoRA on `q,k,v,o,gate,up,down`; cosine schedule; max grad-norm 1.0; no weight decay; seed 42; `eval_on_start`; timed full-state checkpoints pushed to a private Hugging Face repo for provider hopping. r1: one H200 (Lightning AI), gradient checkpointing on, 2.05 h. r2: one RTX PRO 6000 96 GB (Colab), gradient checkpointing off, 11.7 h, 2,118 steps; intermediate forgetting-monitor value 0.0707 at step 800. Merge: `-merge` writes the merged bf16 model with NVARC's extra loader files, uploaded via `kagglehub`.

Grid statistics used in §4–§6. Computed on the first demonstration input/output of each task: public evaluation (120 tasks) mean 5.37 colours, 351 cells, 68% same-shape; public training (1,000 tasks) mean 3.84 colours, 161 cells, 67% same-shape; selected ARC-Mega tasks (first batch) mean 6.6 colours, 501 cells, 55% same-shape.

Noise. Three stock-checkpoint draws under successive scaffold versions scored 28.89, 28.47 and 28.19 (SD 0.35); the two r1 draws differ by 2.08. An unmodified copy of the original public NVARC notebook, without the deadline scaffold, scored 31.53 in one draw at the start of the competition; all comparisons in this paper are between notebooks that share the scaffold.